BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN



BÀI TẬP LỚN
NHẬP MÔN KHOA HỌC DỮ LIỆU

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG GỢI Ý TUẦN TỰ TÍCH HỢP TĂNG CƯỜNG DỮ LIỆU

NGÀNH: KHOA HỌC MÁY TÍNH

CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

SINH VIÊN: **NGUYỄN TRẦN KIÊN**

MÃ SINH VIÊN: **12423018**

MÃ LỚP: **12423TN**

GIẢNG VIÊN HƯỚNG DẪN: **PGS.TS. NGUYỄN MINH TIẾN**

HƯNG YÊN – 2026

NHẬN XÉT

Nhận xét của giảng viên hướng dẫn:

……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

PSG.TS. Nguyễn Minh Tiến

LỜI CAM ĐOAN

Em xin cam đoan dự án " Thiết kế và triển khai hệ thống gợi ý tuần tự tích hợp tăng cường dữ liệu" là kết quả thực hiện của bản thân em dưới sự hướng dẫn của thầy Nguyễn Minh Tiến.

Các tài liệu tham khảo sử dụng trong đồ án đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong đồ án và chương trình xây dựng hoàn toàn là kết quả thực hiện của riêng em.

Nếu vi phạm cam kết này, em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng yên, ngày 19 tháng 06 năm 2026

Sinh viên

Kiên

Nguyễn Trần Kiên

LỜI CẢM ƠN

Để có thể hoàn thành đồ án này, lời đầu tiên em xin phép gửi lời cảm ơn tới bộ môn Khoa học máy tính, Khoa Công nghệ thông tin – Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho em thực hiện đồ án môn học này.

Đặc biệt em xin chân thành cảm ơn thầy Nguyễn Minh Tiến đã rất tận tình hướng dẫn, chỉ bảo em trong suốt thời gian thực hiện đồ án vừa qua.

Em cũng xin chân thành cảm ơn tất cả các Thầy, các Cô trong Trường đã tận tình giảng dạy, trang bị cho em những kiến thức cần thiết, quý báu để giúp em thực hiện được đồ án này.

Mặc dù em đã có cố gắng, nhưng với trình độ còn hạn chế, trong quá trình thực hiện đề tài không tránh khỏi những thiếu sót. Em hy vọng sẽ nhận được những ý kiến nhận xét, góp ý của các Thầy cô về những kết quả triển khai trong đồ án.

Em xin trân trọng cảm ơn!

MỤC LỤC

NHẬN XÉT 2
LỜI CAM ĐOAN 3
LỜI CẢM ƠN 4
MỤC LỤC 5
DANH MỤC CÁC THUẬT NGỮ 7
DANH MỤC CÁC BẢNG 8
DANH MỤC HÌNH ẢNH 9
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 10
1.1. Lý do chọn đề tài 10
1.2. Mục tiêu của đề tài 11
1.3. Giới hạn và phạm vi của đề tài 11
1.3.1. Đối tượng nghiên cứu 11
1.3.2. Phạm vi nghiên cứu 11
1.4. Nội dung thực hiện 12
1.5. Phương pháp tiếp cận 12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 13
2.1. Khái niệm về khai phá dữ liệu và học máy 13
2.1.1. Khái niệm khai phá dữ liệu 13
2.1.2. Khái niệm học máy 13
2.1.3. Liên hệ giữa khai phá dữ liệu và học máy 13
2.2. Các kỹ thuật khai phá dữ liệu 14
2.2.1. Kỹ thuật tiền xử lý dữ liệu 14
2.2.2. Phân tích thống kê dữ liệu 14
2.2.3. Mô hình hóa dữ liệu 14
2.3. Các thuật toán học máy 15
2.3.1. Mô hình popularity 15
2.3.2. Mô hình LSTM (Long Short-Term Memory) 16
2.3.3. Mô hình GRU (Gated Recurrent Unit) 18
2.3.4. Sequential Recommendation 19
2.3.5. Data Augmentation trong Recommendation 19

CHƯƠNG 3: XÂY DỰNG MÔ HÌNH HỌC MÁY 21
3.1. Thu thập dữ liệu 21
3.2. Phân tích và tiền xử lý dữ liệu 22
3.2.1. Phân tích dữ liệu 22
3.2.2. Tiền xử lý dữ liệu 24
3.3. Xây dựng mô hình học máy 26
3.3.1. Tạo dữ liệu huấn luyện 26
3.3.2. Xây dựng Dataset và DataLoader 27
3.3.3. Tinh chỉnh siêu tham số 28
3.3.4. Popularity Baseline 30
3.3.5. Xây dựng mô hình LSTM 31
3.3.6. Xây dựng mô hình GRU 32
3.3.7. Đánh giá mô hình 32
3.3.8. Áp dụng Data Augmentation 33
3.3.9. Huấn luyện và so sánh các phương pháp 33
3.4. Phân tích kết quả thực nghiệm 34
3.4.1. Kết quả mô hình baseline 34
3.4.2. Kết quả sau khi áp dụng Data Augmentation 35
3.4.3. So sánh và đánh giá 35
3.4.4. Phân tích nguyên nhân 35
3.4.5. Hạn chế của mô hình 36
3.5. Kết chương 36
CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG KHAI PHÁ DỮ LIỆU 37
4.1. Triển khai các chức năng nghiệp vụ 37
4.2. Kiểm thử đơn vị và triển khai hệ thống 39
4.3. Kết chương 41
TÀI LIỆU THAM KHẢO 42

DANH MỤC CÁC THUẬT NGỮ

STT	Từ viết tắt	Cụm từ tiếng anh	Diễn giải
1	LSTM	Long-short term memory	Mô hình mạng nơ-ron hồi tiếp có khả năng ghi nhớ thông tin dài hạn trong dữ liệu chuỗi
2	GRU	Gated Recurrent Unit	Biến thể của RNN giúp xử lý dữ liệu chuỗi với cấu trúc đơn giản hơn LSTM
3	RNN	Recurrent Neural Network	Mạng nơ-ron hồi tiếp dùng cho dữ liệu tuần tự
4	HR@10	Hit Rate at 10	Chỉ số đánh giá khả năng gợi ý đúng trong top 10 sản phẩm
5	NDCG@10	Normalized Discounted Cumulative Gain at 10	Chỉ số đánh giá chất lượng và thứ tự của kết quả gợi ý trong top 10

DANH MỤC CÁC BẢNG

Bảng 3. 1: Kết quả tinh chỉnh độ dài chuỗi 28
Bảng 3. 2: Kết quả cho tinh chỉnh siêu tham số 29
Bảng 3. 3: Kết quả thực nghiệm của các mô hình 33
Bảng 3. 4: Kết quả thực nghiệm giữa baseline với Data augmentation 34

DANH MỤC HÌNH ẢNH

Hình 3. 1: Phân bố hành vi người dùng 22
Hình 3. 2: Top 10 item có lượt tương tác nhiều nhất 23
Hình 3. 3: Top 10 sản phẩm được mua nhiều nhất 23
Hình 3. 4: Phân bố độ dài chuỗi hành vi người dùng 24
Hình 4. 1: Giao diện trang đăng nhập 37
Hình 4. 2: Giao diện trang đăng ký 37
Hình 4. 3: Giao diện trang chủ 38
Hình 4. 4: Giao diện trang sản phẩm 38
Hình 4. 5: Giao diện trang gợi ý 38
Hình 4. 6: Giao diện trang giỏ hàng 39
Hình 4. 7: Giao diện trang lịch sử mua hàng 39

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, các hệ thống thương mại điện tử phát triển mạnh mẽ với số lượng sản phẩm ngày càng lớn và đa dạng. Điều này mang lại nhiều lựa chọn cho người dùng, nhưng đồng thời cũng tạo ra một vấn đề quan trọng: người dùng gặp khó khăn trong việc tìm kiếm và lựa chọn sản phẩm phù hợp với nhu cầu cá nhân. Nếu không có sự hỗ trợ, người dùng phải mất nhiều thời gian duyệt qua hàng trăm, hàng nghìn sản phẩm, dẫn đến trải nghiệm không tốt và giảm khả năng mua hàng.

Để giải quyết vấn đề này, các hệ thống gợi ý sản phẩm (Recommender Systems) đã được ứng dụng rộng rãi [6,7,8]. Những hệ thống này giúp đề xuất các sản phẩm phù hợp dựa trên hành vi và sở thích của người dùng, từ đó nâng cao trải nghiệm người dùng và tăng hiệu quả kinh doanh cho nền tảng.

Trong số các phương pháp gợi ý, Sequential Recommendation là một hướng tiếp cận quan trọng [21, 23, 24], vì nó không chỉ xét các sản phẩm mà người dùng đã tương tác, mà còn quan tâm đến thứ tự hành vi theo thời gian. Ví dụ, việc người dùng xem "điện thoại → tai nghe → sạc" sẽ mang ý nghĩa khác so với một thứ tự ngẫu nhiên. Do đó, mô hình gợi ý theo chuỗi có khả năng nắm bắt tốt hơn xu hướng và ý định của người dùng.

Một vấn đề lớn của Sequential Recommendation là dữ liệu hành vi thường bị hạn chế. Trong thực tế, mỗi người dùng chỉ có một số lượng tương tác nhất định, dẫn đến dữ liệu không đủ phong phú để mô hình học hiệu quả.

Để khắc phục hạn chế này, Data Augmentation (tăng cường dữ liệu) được xem là một giải pháp tiềm năng [22, 25, 26, 27]. Phương pháp này tạo ra các biến thể mới từ dữ liệu gốc (ví dụ: cắt chuỗi, đảo vị trí, che một số phần tử, thay thế sản phẩm tương tự…), giúp mở rộng tập dữ liệu mà không cần thu thập thêm dữ liệu thực tế. Nhờ đó, mô hình có thể học được nhiều mẫu hành vi đa dạng hơn, cải thiện độ chính xác và tính ổn định.

Xuất phát từ những lý do trên, đề tài "Thiết kế và triển khai hệ thống gợi ý tuần tự tích hợp tăng cường dữ liệu" được lựa chọn nhằm: Kết hợp giữa mô hình học

máy và ứng dụng web thực tế để nghiên cứu ảnh hưởng của Data Augmentation đến hiệu quả gợi ý và từ đó xây dựng một hệ thống mô phỏng thực tế, trong đó người dùng tương tác và nhận được gợi ý sản phẩm

Đề tài không chỉ có ý nghĩa về mặt học thuật mà còn mang tính ứng dụng cao, giúp hiểu rõ hơn cách các hệ thống gợi ý hoạt động trong các nền tảng thương mại điện tử hiện đại.

1.2. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống gợi ý sản phẩm dựa trên hành vi người dùng, đồng thời nghiên cứu việc áp dụng các kỹ thuật tăng cường dữ liệu để cải thiện hiệu quả của mô hình.

Cụ thể, đề tài hướng tới các mục tiêu sau:

- Tìm hiểu về bài toán Sequential Recommendation và các mô hình học máy thường được sử dụng, đặc biệt là mô hình LSTM trong xử lý dữ liệu chuỗi.
- Nghiên cứu các phương pháp Data Augmentation nhằm tăng cường dữ liệu huấn luyện cho mô hình.
- Xây dựng mô hình gợi ý sản phẩm dựa trên dữ liệu hành vi người dùng thu thập được từ dataset.
- Xây dựng một Web App đơn giản để hiển thị sản phẩm và minh họa kết quả gợi ý, giúp người dùng có thể tương tác trực tiếp với hệ thống.

1.3. Giới hạn và phạm vi của đề tài

1.3.1. Đối tượng nghiên cứu

Hệ thống gợi ý sản phẩm tuần tự (Sequential Recommendation System) ứng dụng các mô hình học máy nhằm dự đoán và đề xuất sản phẩm phù hợp dựa trên hành vi tương tác của người dùng.

1.3.2. Phạm vi nghiên cứu

Phạm vi không gian**:** Hệ thống được xây dựng và triển khai dưới dạng ứng dụng web phục vụ mục đích nghiên cứu, minh họa và thử nghiệm mô hình gợi ý sản phẩm theo chuỗi

Thực tiễn của đề tài**:** Hệ thống có thể được ứng dụng trong các nền tảng thương mại điện tử nhằm hỗ trợ cá nhân hóa trải nghiệm người dùng, nâng cao khả năng gợi ý sản phẩm và tăng hiệu quả tương tác giữa người dùng với hệ thống. Ngoài ra, Web App được xây dựng giúp minh họa trực quan quá trình hoạt động của hệ thống gợi ý và hỗ trợ việc thử nghiệm thực tế.

1.4. Nội dung thực hiện

1. Thu thập dữ liệu
2. Tiền xử lý dữ liệu
3. Phân tích dữ liệu
4. Chuẩn bị dữ liệu huấn luyện
5. Tinh chỉnh tham số tiền xử lý chuỗi
6. Xây dựng dữ liệu chính thức
7. Tinh chỉnh siêu tham số mô hình
8. Huấn luyện và so sánh các mô hình baseline
9. Áp dụng Data Augmentation
10. Huấn luyện mô hình sau augmentation
11. Đánh giá và trực quan hóa kết quả
12. Xây dựng Web App demo

1.5. Phương pháp tiếp cận

1. Tìm và chọn dataset trên kaggle
2. Tìm hiểu các mô hình áp dụng cho bài toán

3 Xây dựng mô hình Sequential Recommendation [21, 23, 24]bằng LSTM/GRU

4 Áp dụng các kỹ thuật Data Augmentation để tăng cường dữ liệu huấn luyện

5 Đánh giá hiệu quả mô hình bằng các chỉ số như HR@10 và NDCG@10 [28, 10]

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Khái niệm về khai phá dữ liệu và học máy

2.1.1. Khái niệm khai phá dữ liệu

Khai phá dữ liệu (Data Mining) là quá trình khám phá các tri thức, mẫu [1] (pattern) hoặc thông tin có giá trị từ một tập dữ liệu lớn. Quá trình này sử dụng các kỹ thuật từ nhiều lĩnh vực như thống kê, học máy và cơ sở dữ liệu nhằm trích xuất những thông tin hữu ích mà con người khó có thể nhận biết bằng phương pháp thủ công.

Trong bối cảnh thương mại điện tử, khai phá dữ liệu đóng vai trò quan trọng trong việc phân tích hành vi người dùng, từ đó giúp doanh nghiệp hiểu được nhu cầu, sở thích và xu hướng mua sắm của khách hàng. Các ứng dụng phổ biến của khai phá dữ liệu bao gồm: phân tích giỏ hàng (market basket analysis), phân cụm khách hàng, dự đoán xu hướng và đặc biệt là hệ thống gợi ý sản phẩm.

2.1.2. Khái niệm học máy

Học máy (Machine Learning) là một nhánh của trí tuệ nhân tạo [3, 4, 5], cho phép máy tính học từ dữ liệu và cải thiện hiệu suất theo thời gian mà không cần lập trình cụ thể cho từng trường hợp.

Thay vì xây dựng các quy tắc cố định, mô hình học máy sẽ được huấn luyện trên dữ liệu để tìm ra mối quan hệ giữa đầu vào và đầu ra. Sau khi huấn luyện, mô hình có thể dự đoán hoặc đưa ra quyết định khi gặp dữ liệu mới.

Một số loại học máy phổ biến gồm:

- Học có giám sát (Supervised Learning)
- Học không giám sát (Unsupervised Learning)
- Học tăng cường (Reinforcement Learning)

2.1.3. Liên hệ giữa khai phá dữ liệu và học máy

Khai phá dữ liệu và học máy có mối quan hệ chặt chẽ với nhau [1, 2]. Khai phá dữ liệu là quá trình tổng thể bao gồm nhiều bước như thu thập dữ liệu, xử lý dữ liệu và phân tích dữ liệu. Trong đó, học máy đóng vai trò là công cụ quan trọng để xây dựng các mô hình phân tích.

Học máy là một phần trong khai phá dữ liệu, giúp tự động hóa quá trình phát hiện các quy luật trong dữ liệu. Trong đề tài này, dữ liệu hành vi người dùng sẽ được khai thác và sử dụng để huấn luyện mô hình học máy nhằm phục vụ cho hệ thống gợi ý.

2.2. Các kỹ thuật khai phá dữ liệu

2.2.1. Kỹ thuật tiền xử lý dữ liệu

Tiền xử lý dữ liệu là bước đầu tiên và rất quan trọng trong quá trình khai phá [1, 2] dữ liệu. Dữ liệu ban đầu thường bị thiếu, nhiễu hoặc không đồng nhất, vì vậy cần được xử lý trước khi đưa vào mô hình.

Một số công việc trong bước này gồm:

- Làm sạch dữ liệu (loại bỏ dữ liệu lỗi, trùng lặp)
- Chuẩn hóa dữ liệu
- Chuyển đổi dữ liệu về dạng phù hợp
- Xây dựng dữ liệu đầu vào cho mô hình

Mục tiêu là đảm bảo dữ liệu đầu vào đạt được độ chính xác và nhất quán cao nhất, từ đó hỗ trợ các công cụ và mô hình phân tích hoạt động hiệu quả.

Hiểu rõ tiền xử lý dữ liệu không chỉ giúp tối ưu hóa quy trình xử lý dữ liệu mà còn đóng vai trò quan trọng trong việc đưa ra những kết quả phân tích đáng tin cậy và có giá trị thực tiễn cao.

2.2.2. Phân tích thống kê dữ liệu

Phân tích thống kê giúp hiểu rõ đặc điểm của dữ liệu trước khi xây dựng mô hình [1, 4].

Các kỹ thuật thống kê thường được sử dụng bao gồm:

- Tính toán các giá trị trung bình, phương sai
- Phân bố tần suất của các sản phẩm
- Phân tích số lượng hành vi của mỗi người dùng

Việc phân tích này giúp phát hiện các vấn đề như dữ liệu mất cân bằng hoặc thiếu đa dạng, từ đó có hướng xử lý phù hợp như áp dụng Data Augmentation.

2.2.3. Mô hình hóa dữ liệu

Mô hình hóa dữ liệu là quá trình xây dựng một cách biểu diễn đơn giản của hệ thống dữ liệu [1], nhằm mô tả các thành phần dữ liệu, thuộc tính của chúng và mối

quan hệ giữa các thành phần đó. Việc mô hình hóa thường được thể hiện dưới dạng sơ đồ hoặc ký hiệu, giúp dễ hình dung cách dữ liệu được lưu trữ và luân chuyển trong hệ thống.

Mô hình dữ liệu đóng vai trò như một bản thiết kế cho cơ sở dữ liệu, hỗ trợ việc xây dựng hệ thống mới hoặc cải tiến hệ thống cũ. Thông qua mô hình hóa dữ liệu, người phát triển có thể xác định rõ các yêu cầu về dữ liệu, từ đó hạn chế sai sót trước khi tiến hành lập trình.

Ngoài ra, mô hình hóa dữ liệu còn giúp chuẩn hóa cách tổ chức và quản lý dữ liệu, đảm bảo dữ liệu được sử dụng một cách nhất quán và hiệu quả. Việc này đặc biệt quan trọng trong các hệ thống có dữ liệu lớn, giúp việc lưu trữ, truy xuất và xử lý dữ liệu trở nên dễ dàng hơn.

Trong thực tế, mô hình hóa dữ liệu không chỉ dành cho các chuyên gia về cơ sở dữ liệu mà còn cần thiết đối với các nhà phân tích và nhà phát triển, đặc biệt trong các bài toán liên quan đến phân tích dữ liệu và học máy. Một mô hình dữ liệu được thiết kế tốt sẽ giúp hệ thống hoạt động ổn định hơn và đáp ứng tốt hơn các yêu cầu của bài toán.

2.3. Các thuật toán học máy

Trong đề tài này, các thuật toán học máy được sử dụng chủ yếu thuộc nhóm mô hình xử lý dữ liệu chuỗi, trong đó tiêu biểu là LSTM và GRU. Đây là các mô hình học sâu có khả năng khai thác thông tin theo thứ tự thời gian, phù hợp với bài toán gợi ý sản phẩm dựa trên hành vi người dùng.

2.3.1. Mô hình popularity

Popularity Baseline là mô hình gợi ý đơn giản dựa trên mức độ phổ biến của sản phẩm trong dữ liệu huấn luyện [6, 10]. Mô hình này không xét đến thứ tự hành vi của từng người dùng như LSTM hoặc GRU, mà tính điểm cho mỗi sản phẩm dựa trên số lần sản phẩm đó xuất hiện trong toàn bộ tập tương tác.

Giả sử tập người dùng là U, tập sản phẩm là I, và chuỗi hành vi của người dùng u được ký hiệu là:

$$S_u = \left[i_1^u, i_2^u, \dots, i_{\{T_u\}}^u\right] \quad (2.1)$$

Trong đó, i_t^u là sản phẩm mà người dùng u tương tác tại thời điểm t, và T_u là độ dài chuỗi hành vi của người dùng đó.

Điểm phổ biến của sản phẩm i được tính bằng tổng số lần sản phẩm đó xuất hiện trong toàn bộ dữ liệu huấn luyện:

$$score(i) = \sum_{u \in U} \sum_{t=1}^{T_u} I(i_t^u = i) \quad (2.2)$$

Trong đó, $I(i_t^u = i)$ là hàm chỉ báo:

$$I(i_t\hat{}u = i) = \begin{cases} 1, \text{nếu } i_t^u = i \\ 0, \text{nếu } i_t^u \neq i \end{cases} \quad (2.3)$$

Sản phẩm có score(i) càng lớn thì càng được xem là phổ biến và có khả năng được gợi ý cao hơn. Có thể chuẩn hóa điểm phổ biến thành xác suất tương đối như sau:

$$P_{(pop)(i)} = \frac{score(i)}{\sum_{j \in I} score(j)} \quad (2.4)$$

Khi tạo danh sách gợi ý cho người dùng u, mô hình chọn ra K sản phẩm có điểm phổ biến cao nhất:

$$R_u^K = \underset{(i \in I \setminus S_u)}{TopK}\big(score(i)\big) \quad (2.5)$$

Trong đó, R_u^K là danh sách K sản phẩm được gợi ý cho người dùng u. Việc loại bỏ các sản phẩm thuộc S_u giúp hệ thống tránh gợi ý lại những sản phẩm mà người dùng đã xem hoặc đã mua.

Mặc dù đơn giản, Popularity Baseline thường được sử dụng làm mô hình so sánh cơ bản trong hệ thống gợi ý. Nếu mô hình LSTM hoặc GRU cho kết quả tốt hơn Popularity Baseline, điều đó cho thấy mô hình học sâu đã khai thác được thông tin tuần tự trong hành vi người dùng thay vì chỉ dựa vào độ phổ biến chung của sản phẩm.

2.3.2. Mô hình LSTM (Long Short-Term Memory)

LSTM là một loại mạng nơ-ron hồi tiếp (Recurrent Neural Network – RNN), được thiết kế để giải quyết vấn đề mất thông tin khi xử lý các chuỗi dữ liệu dài [17].

Trong các RNN truyền thống [16], thông tin ở các bước trước thường bị "quên" khi chuỗi quá dài, làm giảm hiệu quả của mô hình.

Để khắc phục điều này, LSTM sử dụng một cơ chế đặc biệt gồm các "cổng" (gate) để kiểm soát luồng thông tin [17, 18].:

- Cổng quên (Forget gate): quyết định thông tin nào từ quá khứ cần được giữ lại hoặc loại bỏ
- Cổng vào (Input gate): quyết định thông tin mới nào được thêm vào bộ nhớ
- Cổng đầu ra (Output gate): quyết định thông tin nào được sử dụng để đưa ra kết quả

Với đầu vào tại thời điểm t là vector embedding x_t, hidden state trước đó là h_{t-1} và cell scstate trước đó là c_{t-1}, LSTM tính toán các cổng như sau:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{2.6}$$

Trong đó f_t là cổng quên (forget gate). Giá trị của f_t nằm trong khoảng từ 0 đến 1, quyết định lượng thông tin cũ trong c_{t-1} được giữ lại. Nếu một phần tử của f_t gần 0, thông tin tương ứng sẽ bị loại bỏ; nếu gần 1, thông tin đó được giữ lại.

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{2.7}$$

Trong đó i_t là cổng vào (input gate), quyết định lượng thông tin mới sẽ được ghi vào bộ nhớ.

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{2.8}$$

$\tilde{c}_t$ là trạng thái bộ nhớ ứng viên, chứa thông tin mới được tạo ra từ đầu vào hiện tại x_t và trạng thái ẩn trước đó h_{t-1}.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{2.9}$$

Cell state mới c_t được tính bằng cách kết hợp phần thông tin cũ được giữ lại $f_t \odot c_{t-1}$ và phần thông tin mới được thêm vào $i_t \odot \tilde{c}_t$.

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{2.10}$$

Trong đó o_t là cổng đầu ra (output gate), quyết định phần nào của cell state sẽ được dùng để tạo hidden state đầu ra.

$$h_t = o_t \odot \tanh(c_t) \tag{2.11}$$

Hidden state h_t là đầu ra của LSTM tại thời điểm t. Vector này mang thông tin ngữ cảnh của chuỗi đến thời điểm hiện tại và được dùng để dự đoán sản phẩm tiếp theo.

Nhờ cơ chế này, LSTM có khả năng ghi nhớ thông tin trong thời gian dài và học được mối quan hệ phức tạp giữa các phần tử trong chuỗi.

Trong bài toán gợi ý sản phẩm, LSTM được sử dụng để xử lý chuỗi hành vi của người dùng (ví dụ: các sản phẩm đã xem hoặc mua theo thứ tự thời gian), từ đó dự đoán sản phẩm tiếp theo mà người dùng có khả năng quan tâm.

2.3.3. Mô hình GRU (Gated Recurrent Unit)

GRU là một biến thể đơn giản hơn của LSTM, cũng thuộc họ mạng RNN [19, 20]. GRU được thiết kế với mục tiêu giảm độ phức tạp của mô hình nhưng vẫn giữ được hiệu quả trong việc xử lý dữ liệu chuỗi.

Khác với LSTM, GRU chỉ sử dụng hai cổng chính:

- Cổng cập nhật (Update gate): quyết định lượng thông tin từ quá khứ được giữ lại
- Cổng reset (Reset gate): quyết định mức độ bỏ qua thông tin cũ

Với đầu vào tại thời điểm t là x_t và hidden state trước đó là h_{t-1}, GRU tính toán hai cổng chính như sau:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \tag{2.12}$$

Trong đó z_t là cổng cập nhật (update gate), quyết định mức độ giữ lại thông tin quá khứ. Nếu z_t lớn, mô hình ưu tiên cập nhật hidden state bằng thông tin mới; nếu z_t nhỏ, mô hình giữ lại nhiều thông tin từ h_{t-1}.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \tag{2.13}$$

Trong đó r_t là cổng reset (reset gate), quyết định mức độ sử dụng thông tin quá khứ khi tạo hidden state ứng viên. Nếu r_t nhỏ, mô hình giảm ảnh hưởng của trạng thái cũ.

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \tag{2.14}$$

$\tilde{h}_t$ là hidden state ứng viên, được tính từ đầu vào hiện tại và phần thông tin quá khứ đã được điều chỉnh bởi cổng reset.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{2.15}$$

Hidden state mới h_t được tính bằng cách kết hợp hidden state cũ h_{t-1} và hidden state ứng viên $\tilde{h}_t$. Nhờ đó, GRU có thể kiểm soát việc ghi nhớ thông tin quá khứ và cập

nhật thông tin mới mà không cần cell state riêng như LSTM.

Do có cấu trúc đơn giản hơn, GRU có số lượng tham số ít hơn so với LSTM [19, 20], từ đó giúp giảm thời gian huấn luyện và hạn chế hiện tượng quá khớp (overfitting) khi dữ liệu không quá lớn.

Trong một số trường hợp, GRU có thể cho hiệu quả tương đương LSTM nhưng với tốc độ xử lý nhanh hơn, do đó thường được sử dụng như một lựa chọn thay thế phù hợp.

2.3.4. Sequential Recommendation

Sequential Recommendation là phương pháp gợi ý dựa trên thứ tự các hành vi của người dùng theo thời gian [21, 23, 24]. Khác với các phương pháp truyền thống chỉ xét tập hợp các sản phẩm đã tương tác, phương pháp này quan tâm đến trình tự và ngữ cảnh của hành vi.

Giả sử chuỗi hành vi của một người dùng là: Điện thoại → Tai nghe → Sạc Mô hình sẽ học được mối liên hệ giữa các sản phẩm này và dự đoán sản phẩm tiếp theo có thể là:

→ Ốp lưng hoặc pin dự phòng

Về mặt toán học, mô hình học cách ước lượng xác suất của sản phẩm tiếp theo dựa trên các sản phẩm trước đó trong chuỗi. Điều này giúp hệ thống đưa ra các gợi ý phù hợp hơn với nhu cầu thực tế của người dùng tại thời điểm hiện tại.

2.3.5. Data Augmentation trong Recommendation

Data Augmentation là kỹ thuật tạo thêm dữ liệu huấn luyện từ dữ liệu gốc nhằm tăng độ đa dạng và giúp mô hình học tốt hơn [22, 25, 26, 27]. Trong bài toán gợi ý theo chuỗi, dữ liệu thường bị hạn chế hoặc không đủ phong phú, nên việc tăng cường dữ liệu là cần thiết.

Một số phương pháp được sử dụng trong đề tài gồm:

- Crop: cắt bớt một phần của chuỗi hành vi để tạo chuỗi ngắn hơn
- Mask: che một số phần tử trong chuỗi bằng ký hiệu đặc biệt
- Reorder: thay đổi thứ tự của một đoạn nhỏ trong chuỗi
- Substitute: thay thế một sản phẩm bằng sản phẩm tương tự

Ví dụ với chuỗi ban đầu: [A, B, C, D]

Có thể tạo ra các chuỗi mới như:

→ [B, C, D] (Crop)

→ [A, _, C, D] (Mask)

→ [A, C, B, D] (Reorder)

Các kỹ thuật này giúp mô hình học được nhiều biến thể hành vi khác nhau, từ đó cải thiện khả năng dự đoán và tăng hiệu quả của hệ thống gợi ý.

CHƯƠNG 3: XÂY DỰNG MÔ HÌNH HỌC MÁY

3.1. Thu thập dữ liệu

Kaggle: https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset

Bộ dữ liệu sử dụng: Retailrocket recommender system dataset. Đây là bộ dữ liệu phổ biến trong các bài toán gợi ý sản phẩm, bao gồm thông tin về hành vi của người dùng trên một website thương mại điện tử.

Bộ dữ liệu bao gồm ba tệp: một tệp chứa dữ liệu hành vi (events.csv), một tệp chứa thuộc tính mặt hàng (item_properties.csv) và một tệp mô tả cây danh mục (category_tree.csv). Dữ liệu được thu thập từ một trang web thương mại điện tử thực tế. Đây là dữ liệu thô, tức là không qua bất kỳ chuyển đổi nội dung nào, tuy nhiên, tất cả các giá trị đều được mã hóa băm do vấn đề bảo mật. Mục đích công bố là để thúc đẩy các nghiên cứu trong lĩnh vực hệ thống đề xuất với phản hồi ngầm.

Trong đề tài này, file events.csv đóng vai trò quan trọng nhất vì đây là dữ liệu chính dùng để xây dựng chuỗi hành vi người dùng. Các file thuộc tính sản phẩm được sử dụng bổ sung để lấy thông tin danh mục, phục vụ cho phương pháp tăng cường dữ liệu dạng substitute.

Dữ liệu ban đầu là dữ liệu thô, các mã người dùng và mã sản phẩm đều đã được mã hóa nhằm đảm bảo tính bảo mật. Tuy nhiên, dữ liệu vẫn giữ được cấu trúc hành vi theo thời gian, phù hợp với bài toán Sequential Recommendation.

Các thuộc tính chính trong dữ liệu

Trong file events.csv, dữ liệu hành vi người dùng bao gồm các thuộc tính sau:

- visitorid: mã định danh của người dùng
- itemid: mã sản phẩm mà người dùng tương tác
- timestamp: thời điểm xảy ra hành vi (đơn vị millisecond)
- event: loại hành vi, bao gồm: view (xem sản phẩm), addtocart (thêm vào giỏ hàng), transaction(mua hàng)

Trong file item_properties, dữ liệu sản phẩm bao gồm nhiều thuộc tính, trong đó đề tài tập trung sử dụng:

- itemid: mã sản phẩm
- property: tên thuộc tính
- value: giá trị của thuộc tính
- timestamp: thời điểm cập nhật thuộc tính
- categoryid: danh mục của sản phẩm
- available: trạng thái còn hàng

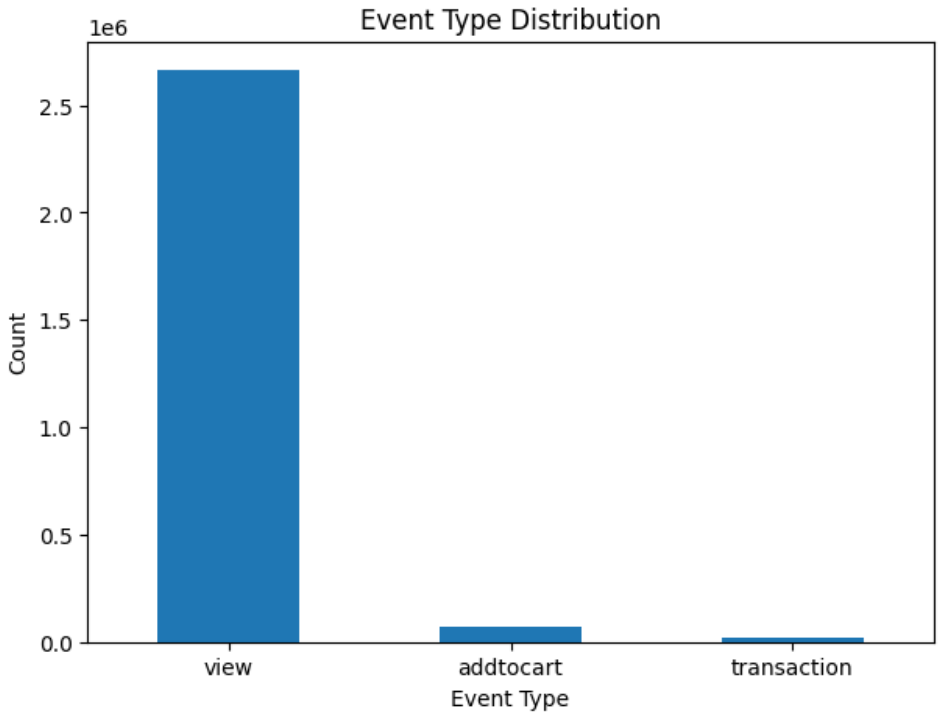
Trong file category_tree.csv, dữ liệu gồm:

- categoryid: mã danh mục
- parentid: danh mục cha

3.2. Phân tích và tiền xử lý dữ liệu

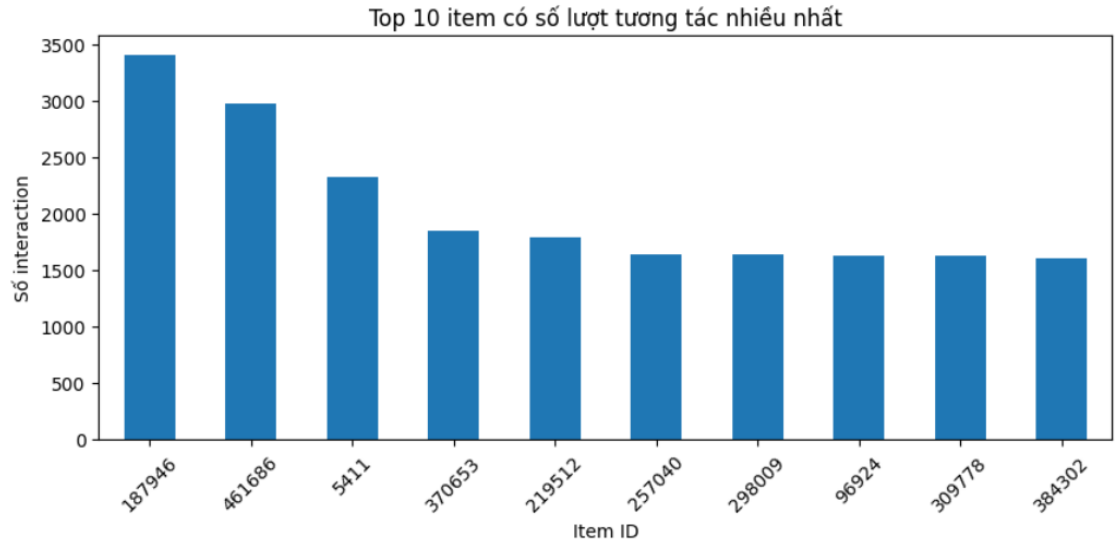
3.2.1. Phân tích dữ liệu

- Phân bố hành vi người dùng



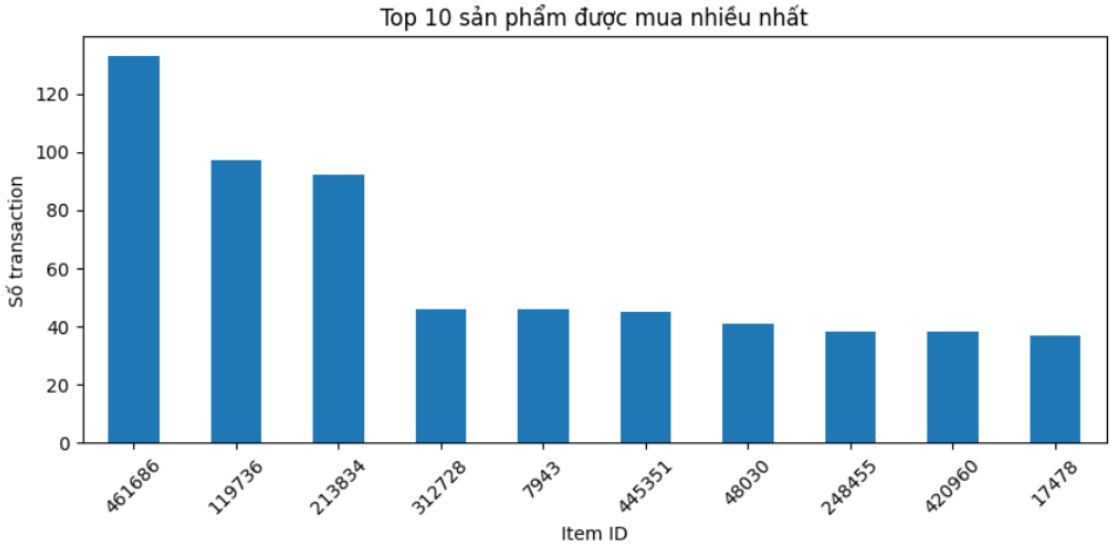
Hình 3. 1: Phân bố hành vi người dùng

- Top 10 item có lượt tương tác nhiều nhất



Hình 3. 2: Top 10 item có lượt tương tác nhiều nhất

- Top 10 sản phẩm được mua nhiều nhất



Hình 3. 3: Top 10 sản phẩm được mua nhiều nhất

- Phân bố chuỗi hành vi người dùng có độ dài chuỗi < 50



Hình 3. 4: Phân bố độ dài chuỗi hành vi người dùng

3.2.2. Tiền xử lý dữ liệu

Dữ liệu trong đề tài được đọc từ các file bao gồm events.csv, item_properties_part1.csv, item_properties_part2.csv và category_tree.csv. Trong đó, file events.csv chứa thông tin hành vi người dùng và là nguồn dữ liệu chính để xây dựng mô hình. Bảng events.csv ban đầu bao gồm các cột: timestamp, visitorid, event, itemid và transactionid.

Tuy nhiên, nhận thấy cột transactionid chỉ xuất hiện ở các hành vi mua hàng và có rất nhiều giá trị thiếu (NaN). Điều này khiến cột này không mang nhiều ý nghĩa trong việc xây dựng chuỗi hành vi người dùng.

Do đó, cột transactionid được loại bỏ, và chỉ giữ lại các cột quan trọng gồm visitorid, itemid, timestamp và event. Đây là các thông tin cốt lõi để mô hình có thể học được hành vi người dùng theo thời gian.

Ngược lại, các cột visitorid, itemid, timestamp và event được giữ lại vì:

- visitorid dùng để phân biệt người dùng

- itemid xác định sản phẩm được tương tác
- timestamp giúp xác định thứ tự thời gian
- event thể hiện loại hành vi

Đây là các thông tin cốt lõi để xây dựng chuỗi hành vi người dùng trong bài toán Sequential Recommendation.
Sau khi lựa chọn các cột cần thiết từ dữ liệu hành vi, bước tiếp theo là tiến hành xử lý dữ liệu thuộc tính sản phẩm từ hai file item_properties

Trong dữ liệu ban đầu, mỗi sản phẩm có thể có nhiều thuộc tính khác nhau, được lưu dưới dạng nhiều dòng, trong đó mỗi dòng biểu diễn một thuộc tính tại một thời điểm nhất định. Tuy nhiên, không phải tất cả các thuộc tính đều cần thiết cho bài toán gợi ý theo chuỗi.

Trong phạm vi đề tài, chỉ giữ lại hai thuộc tính quan trọng là categoryid và available. Thuộc tính categoryid được sử dụng để xác định danh mục của sản phẩm, phục vụ cho việc phân tích và đặc biệt là áp dụng phương pháp tăng cường dữ liệu dạng thay thế (substitute). Thuộc tính available cho biết trạng thái tồn tại của sản phẩm, giúp đảm bảo rằng các sản phẩm không hợp lệ có thể được xử lý phù hợp.

Các thuộc tính khác bị loại bỏ nhằm giảm độ phức tạp của dữ liệu và tránh gây nhiễu cho mô hình, do không liên quan trực tiếp đến hành vi tuần tự của người dùng.

Sau khi lọc thuộc tính, hai file dữ liệu sản phẩm được gộp lại thành một bảng duy nhất. Việc gộp dữ liệu là cần thiết vì thông tin sản phẩm được phân tán ở nhiều file khác nhau, nếu không gộp sẽ dẫn đến thiếu dữ liệu trong quá trình xử lý.

Dữ liệu thuộc tính sản phẩm được xử lý theo thời gian. Do một sản phẩm có thể có nhiều bản ghi thuộc tính tại các thời điểm khác nhau, nên cần sắp xếp dữ liệu theo timestamp và chỉ giữ lại giá trị cuối cùng của mỗi thuộc tính cho từng sản phẩm.

Việc này giúp đảm bảo rằng thông tin được sử dụng là thông tin cập nhật nhất, đồng thời tránh trùng lặp dữ liệu.

Sau đó, dữ liệu được chuyển từ dạng nhiều dòng sang dạng bảng bằng thao tác pivot, trong đó mỗi dòng tương ứng với một sản phẩm, và các thuộc tính như categoryid và available trở thành các cột.

Dữ liệu hành vi người dùng được ghép với dữ liệu sản phẩm thông qua khóa itemid. Việc ghép dữ liệu giúp mỗi hành vi của người dùng có thêm thông tin về danh mục sản phẩm.

Sau khi ghép, mỗi dòng dữ liệu không chỉ chứa thông tin hành vi mà còn có thêm các thuộc tính liên quan đến sản phẩm, giúp mô hình có thêm thông tin để học. Trong quá trình ghép dữ liệu, một số sản phẩm không có thông tin về categoryid hoặc available, dẫn đến xuất hiện các giá trị thiếu. Để xử lý vấn đề này, các giá trị thiếu được thay thế bằng giá trị -1.

→ Việc thay thế giá trị thiếu giúp tránh lỗi trong quá trình huấn luyện mô hình, đồng thời vẫn giữ được cấu trúc dữ liệu dạng số.

Cuối cùng, dữ liệu được nhóm theo từng người dùng để tạo thành chuỗi hành vi. Mỗi người dùng sẽ được biểu diễn dưới dạng một danh sách các sản phẩm đã tương tác theo thứ tự thời gian. Các chuỗi này chính là đầu vào của mô hình LSTM trong bài toán gợi ý sản phẩm.

3.3. Xây dựng mô hình học máy

Sau khi hoàn tất quá trình tiền xử lý, dữ liệu đã được chuyển thành dạng chuỗi hành vi người dùng. Đây là dạng dữ liệu phù hợp để đưa vào các mô hình học sâu xử lý chuỗi, đặc biệt là LSTM. Trong phần này, tiến hành xây dựng mô hình học máy để dự đoán sản phẩm tiếp theo mà người dùng có khả năng tương tác.

3.3.1. Tạo dữ liệu huấn luyện

Từ các chuỗi hành vi người dùng đã xây dựng ở phần trước, dữ liệu được chuyển đổi thành các cặp đầu vào và đầu ra phục vụ cho việc huấn luyện mô hình.

Cụ thể, với mỗi chuỗi hành vi, một phần của chuỗi được sử dụng làm đầu vào (input), và phần tử tiếp theo trong chuỗi được sử dụng làm nhãn dự đoán (target).

Ví dụ, với chuỗi hành vi: **[A,B,C,D,E]**

có thể tạo ra các cặp dữ liệu như sau:

- **Input: [A→ Target: B**
- **Input: [A,B] → Target: C**
- **Input: [A,B,C] → Target: D**

Để đảm bảo mô hình có thể học được đầy đủ các mối quan hệ trong chuỗi, các chuỗi có độ dài từ 3 trở lên được sử dụng để tạo dữ liệu huấn luyện. Đồng thời, dữ liệu cũng được chia thành ba tập:

- Tập huấn luyện (train): dùng để huấn luyện mô hình
- Tập validation: dùng để đánh giá trong quá trình thử nghiệm
- Tập kiểm tra (test): dùng để đánh giá cuối cùng

Ngoài ra, độ dài tối đa của chuỗi đầu vào được giới hạn là 20 (max_len = 20). Nếu chuỗi dài hơn, chỉ lấy 20 phần tử gần nhất [21, 23], vì các hành vi gần thời điểm hiện tại thường có ảnh hưởng lớn hơn đến hành vi tiếp theo.

3.3.2. Xây dựng Dataset và DataLoader

Để đưa dữ liệu vào mô hình LSTM trong PyTorch [30], dữ liệu được đóng gói dưới dạng Dataset thông qua lớp SimpleSeqDataset.

Trong lớp này, mỗi mẫu dữ liệu bao gồm một chuỗi đầu vào và một nhãn dự đoán. Nếu chuỗi đầu vào dài hơn max_len, chuỗi sẽ được cắt bớt, chỉ giữ lại phần cuối.

Do các chuỗi có độ dài khác nhau, cần thực hiện padding để đưa các chuỗi trong cùng một batch về cùng độ dài. Trong đề tài, giá trị padding được đặt là 0, tương ứng với giá trị không có sản phẩm.

Quá trình padding được thực hiện thông qua hàm collate_fn, giúp chuẩn hóa dữ liệu trước khi đưa vào mô hình.

Sau đó, DataLoader được sử dụng để chia dữ liệu thành các batch nhỏ, giúp quá trình huấn luyện diễn ra hiệu quả hơn.

3.3.3. Tinh chỉnh siêu tham số

Trước khi xây dựng mô hình chính thức, đề tài tiến hành thử nghiệm và điều chỉnh một số siêu tham số nhằm lựa chọn cấu hình phù hợp nhất cho bài toán Sequential Recommendation.

Các siêu tham số được thử nghiệm gồm:

- embed_dim: kích thước vector embedding
- hidden_dim: kích thước hidden state của LSTM
- learning_rate: tốc độ học
- dropout: tỉ lệ dropout
- min_seq_len: độ dài chuỗi tối thiểu
- max_seq_len: độ dài chuỗi tối đa

a) Tinh chỉnh độ dài chuỗi

Đề tài tiến hành thử nghiệm nhiều giá trị khác nhau cho min_seq_len và max_seq_len nhằm xác định độ dài chuỗi phù hợp cho mô hình.

Việc giới hạn độ dài chuỗi giúp giảm nhiễu từ các hành vi quá xa trong quá khứ, đồng thời giảm chi phí tính toán trong quá trình huấn luyện.

Bảng 3. 1: Kết quả tinh chỉnh độ dài chuỗi

min_seq_len	max_seq_len	val_HR@10	val_NDCG@10
5	**20**	**0.239895**	**0.199012**
5	50	0.225733	0.186734
5	100	0.221121	0.181701
3	20	0.155137	0.128260
2	20	0.150266	0.123598
2	50	0.141606	0.116217
3	50	0.140609	0.115964
3	100	0.136193	0.110638

2	100	0.132914	0.108437

b) Tinh chỉnh embedding dimension

Các giá trị thử nghiệm:

- embed_dim = 64
- embed_dim = 128

c) Tinh chỉnh hidden dimension

Các giá trị thử nghiệm:

- hidden_dim = 64
- hidden_dim = 128

d) Tinh chỉnh learning rate

Các giá trị thử nghiệm:

- learning_rate = 0.001
- learning_rate = 0.0005

e) Tinh chỉnh dropout

Các giá trị thử nghiệm:

- dropout = 0.1
- dropout = 0.3

Việc sử dụng dropout giúp giảm hiện tượng overfitting [29] trong quá trình huấn luyện mô hình.

Bảng 3. 2: Kết quả cho tinh chỉnh siêu tham số

embed_dim	**hidden_dim**	**learning_rate**	**dropout**	**HR@10**	**NDCG@10**
128	**128**	**0.0010**	**0.1**	**0.444487**	**0.361496**
64	128	0.0010	0.1	0.417263	0.342526

128	128	0.0010	0.3	0.414102	0.339520
64	128	0.0010	0.3	0.385506	0.318065
128	128	0.0005	0.1	0.367888	0.305227
128	64	0.0010	0.1	0.369284	0.303460
64	64	0.0010	0.1	0.343470	0.282283
128	64	0.0010	0.3	0.336658	0.276334
64	128	0.0005	0.1	0.332590	0.276101
128	128	0.0005	0.3	0.330617	0.274437
64	64	0.0010	0.3	0.310132	0.254628
64	128	0.0005	0.3	0.300674	0.249753
128	64	0.0005	0.1	0.265278	0.216588
128	64	0.0005	0.3	0.238557	0.193960
64	64	0.0005	0.1	0.231769	0.187280
64	64	0.0005	0.3	0.209336	0.167496

3.3.4. Popularity Baseline

Để làm cơ sở so sánh cho các mô hình học sâu, đề tài xây dựng mô hình Popularity Baseline. Đây là phương pháp gợi ý đơn giản dựa trên tần suất xuất hiện của sản phẩm trong tập dữ liệu huấn luyện.

Cụ thể, các sản phẩm được sắp xếp theo số lần xuất hiện hoặc số lần tương tác của người dùng. Khi thực hiện gợi ý, hệ thống sẽ đề xuất top-K sản phẩm phổ biến nhất cho tất cả người dùng mà không xét đến ngữ cảnh hay lịch sử hành vi cá nhân.

Mặc dù có cấu trúc đơn giản, Popularity Baseline thường được sử dụng trong các nghiên cứu Recommendation System nhằm:

- Làm mốc so sánh cơ bản
- Đánh giá mức độ cải thiện của các mô hình học sâu
- Kiểm tra tính hiệu quả của mô hình Sequential Recommendation

Tuy nhiên, phương pháp này có hạn chế là không cá nhân hóa gợi ý và không khai thác được mối quan hệ tuần tự giữa các hành vi người dùng.

3.3.5. Xây dựng mô hình LSTM

Mô hình được sử dụng trong đề tài là LSTM (Long Short-Term Memory), một loại mạng nơ-ron hồi tiếp có khả năng xử lý dữ liệu chuỗi và ghi nhớ thông tin dài hạn.

Mô hình bao gồm ba thành phần chính:

- Embedding layer: chuyển đổi mỗi sản phẩm từ dạng số nguyên sang vector đặc trưng có kích thước cố định
- LSTM layer: học mối quan hệ tuần tự giữa các sản phẩm trong chuỗi
- Fully Connected layer: dự đoán xác suất của sản phẩm tiếp theo

Embedding layer được sử dụng để chuyển đổi mỗi item_id sang vector đặc trưng liên tục, trong khi hidden state của LSTM được sử dụng để lưu trữ thông tin ngữ cảnh của chuỗi hành vi người dùng. Các giá trị siêu tham số cụ thể của mô hình được lựa chọn thông qua quá trình tinh chỉnh mô hình ở phần trước.Trong quá trình xử lý, mô hình sử dụng cơ chế mask để xác định độ dài thực của mỗi chuỗi, từ đó áp dụng hàm pack_padded_sequence nhằm loại bỏ ảnh hưởng của các giá trị padding. Điều này giúp mô hình chỉ học trên dữ liệu thực tế, tránh bị nhiễu bởi các phần tử không có ý nghĩa.

Mô hình baseline được huấn luyện trên dữ liệu gốc (chưa áp dụng Data Augmentation) với cấu hình siêu tham số đã được lựa chọn ở phần trước.

Các thiết lập huấn luyện gồm:

- Hàm mất mát: CrossEntropyLoss
- Optimizer: Adam

- learning_rate = 0.001
- embed_dim = 128
- hidden_dim = 128
- dropout = 0.1
- max_seq_len = 20
- min_seq_len = 5

Trong mỗi epoch, dữ liệu được đưa vào mô hình theo từng batch. Mô hình thực hiện dự đoán sản phẩm tiếp theo, sau đó tính toán hàm mất mát, thực hiện lan truyền ngược (backpropagation) và cập nhật trọng số thông qua optimizer Adam.

Để đảm bảo quá trình huấn luyện ổn định, kỹ thuật gradient clipping được sử dụng nhằm giới hạn giá trị gradient, giúp tránh hiện tượng gradient exploding trong quá trình huấn luyện mạng LSTM.

3.3.6. Xây dựng mô hình GRU

Ngoài mô hình LSTM, đề tài cũng tiến hành thử nghiệm mô hình GRU (Gated Recurrent Unit) nhằm so sánh hiệu quả giữa các kiến trúc mạng hồi tiếp trong bài toán Sequential Recommendation.

GRU là một biến thể của RNN có cấu trúc đơn giản hơn LSTM do sử dụng ít cổng hơn, giúp giảm số lượng tham số và tăng tốc độ huấn luyện. Mặc dù đơn giản hơn, GRU vẫn có khả năng học được mối quan hệ tuần tự trong dữ liệu chuỗi.
Cấu trúc mô hình GRU trong đề tài cũng bao gồm:

- Embedding layer
- GRU layer
- Fully Connected layer

Các siêu tham số huấn luyện của GRU được giữ tương tự mô hình LSTM nhằm đảm bảo tính công bằng trong quá trình so sánh.

3.3.7. Đánh giá mô hình

Sau khi huấn luyện, mô hình được đánh giá bằng hai chỉ số phổ biến trong hệ thống gợi ý:

- HR@10 (Hit Rate@10): kiểm tra xem sản phẩm thực tế có nằm trong top 10 sản phẩm được gợi ý hay không

- NDCG@10: đánh giá chất lượng thứ hạng của sản phẩm trong danh sách gợi ý

Bảng 3. 3: Kết quả thực nghiệm của các mô hình

Method	HR@10	NDCG@10
LSTM	**0.458834**	**0.363447**
GRU	0.424945	0.334247
Popularity	0.007853	0.004253

3.3.8. Áp dụng Data Augmentation

Để cải thiện hiệu suất mô hình, đề tài áp dụng các phương pháp tăng cường dữ liệu trên chuỗi hành vi người dùng. Các phương pháp bao gồm:

- Crop: cắt một đoạn con từ chuỗi
- Mask: thay một số phần tử bằng token đặc biệt
- Reorder: thay đổi thứ tự một đoạn nhỏ trong chuỗi
- Substitute: thay thế sản phẩm bằng sản phẩm cùng danh mục

Các phương pháp này giúp tạo ra nhiều biến thể khác nhau của chuỗi hành vi, từ đó tăng độ đa dạng của dữ liệu huấn luyện và giúp mô hình học được nhiều dạng hành vi hơn.

3.3.9. Huấn luyện và so sánh các phương pháp

Để đánh giá hiệu quả của Data Augmentation, mô hình được huấn luyện lại với từng phương pháp khác nhau, bao gồm: không dùng augmentation (baseline), crop, mask, reorder và substitute.

Với mỗi phương pháp, mô hình được huấn luyện độc lập và đánh giá trên cùng tập test. Kết quả được lưu lại để so sánh. Việc so sánh này giúp xác định phương pháp nào mang lại hiệu quả tốt nhất cho bài toán.

Bảng 3. 4: Kết quả thực nghiệm giữa baseline với Data augmentation

Method	**HR@10**	**NDCG@10**
substitute	**0.475999**	**0.373435**
crop	0.470718	0.370377
mask	0.465866	0.367448
reorder	0.464310	0.366357
LSTM Baseline	0.458405	0.362511

3.4. Phân tích kết quả thực nghiệm

Sau khi hoàn thành quá trình huấn luyện mô hình, tiến hành đánh giá và phân tích kết quả nhằm xác định hiệu quả của mô hình cũng như tác động của các phương pháp tăng cường dữ liệu.

3.4.1. Kết quả mô hình baseline

Mô hình LSTM baseline được huấn luyện trên dữ liệu gốc, chưa áp dụng bất kỳ phương pháp Data Augmentation nào. Kết quả đánh giá được tính toán dựa trên hai chỉ số HR@10 và NDCG@10.

Kết quả cho thấy mô hình có khả năng học được mối quan hệ giữa các sản phẩm trong chuỗi hành vi người dùng. Trong nhiều trường hợp, sản phẩm thực tế xuất hiện trong danh sách top 10 sản phẩm được gợi ý, cho thấy mô hình có khả năng dự đoán tương đối tốt.

Tuy nhiên, do dữ liệu hành vi trong bài toán gợi ý thường khá thưa và không đồng đều, mô hình baseline vẫn còn một số hạn chế, đặc biệt trong việc dự đoán các hành vi ít phổ biến.

Từ kết quả trên có thể thấy HR@10 và NDCG@10 đạt ở mức trung bình, phản ánh khả năng học cơ bản của mô hình nhưng chưa thực sự tối ưu.

3.4.2. Kết quả sau khi áp dụng Data Augmentation

Sau khi áp dụng các phương pháp tăng cường dữ liệu như crop, mask, reorder và substitute, mô hình được huấn luyện lại và đánh giá trên cùng tập test.

Kết quả cho thấy hiệu suất của mô hình được cải thiện rõ rệt so với baseline. Việc tạo ra nhiều biến thể của chuỗi hành vi giúp mô hình học được nhiều mẫu dữ liệu hơn, từ đó tăng khả năng tổng quát hóa.

Trong các phương pháp, **mask** và **substitute** thường cho kết quả tốt hơn. Phương pháp mask giúp mô hình học được ngữ cảnh của chuỗi ngay cả khi thiếu một phần thông tin, trong khi substitute tận dụng được mối quan hệ giữa các sản phẩm cùng danh mục

3.4.3. So sánh và đánh giá

Khi so sánh giữa các phương pháp, có thể rút ra một số nhận xét:

- Mô hình baseline có khả năng học chuỗi nhưng còn hạn chế do dữ liệu thưa
- Data Augmentation giúp tăng độ đa dạng của dữ liệu huấn luyện

Điều này cho thấy việc tăng cường dữ liệu là cần thiết trong bài toán gợi ý theo chuỗi, đặc biệt khi dữ liệu thực tế không đầy đủ.

3.4.4. Phân tích nguyên nhân

Nguyên nhân chính dẫn đến sự cải thiện hiệu suất là do Data Augmentation giúp mô hình không bị phụ thuộc quá mức vào một dạng chuỗi cố định.

- Với **mask**, mô hình buộc phải dự đoán dựa trên ngữ cảnh xung quanh
- Với **substitute**, mô hình học được sự tương đồng giữa các sản phẩm
- Với **crop**, mô hình học từ các chuỗi ngắn hơn
- Với **reorder**, mô hình giảm phụ thuộc vào thứ tự tuyệt đối

Nhờ đó, mô hình trở nên linh hoạt hơn và có khả năng dự đoán tốt hơn trong nhiều tình huống khác nhau.

3.4.5. Hạn chế của mô hình

Mặc dù đạt được kết quả tích cực, mô hình vẫn tồn tại một số hạn chế:

- Dữ liệu sử dụng là dữ liệu ẩn danh (hash), không có thông tin ngữ nghĩa sản phẩm
- Chưa tối ưu sâu các siêu tham số như số layer, learning rate
- Chưa thử nghiệm các mô hình phức tạp hơn như Transformer
- Chưa xử lý triệt để vấn đề cold-start (người dùng mới, sản phẩm mới)

3.5. Kết chương

Đề tài đã trình bày toàn bộ quá trình xây dựng mô hình học máy cho bài toán gợi ý sản phẩm dựa trên dữ liệu hành vi người dùng.

Dữ liệu ban đầu được xử lý và chuyển đổi thành chuỗi hành vi theo thời gian, sau đó được sử dụng để huấn luyện mô hình LSTM nhằm dự đoán sản phẩm tiếp theo. Kết quả thực nghiệm cho thấy mô hình có khả năng học được mối quan hệ tuần tự giữa các sản phẩm.

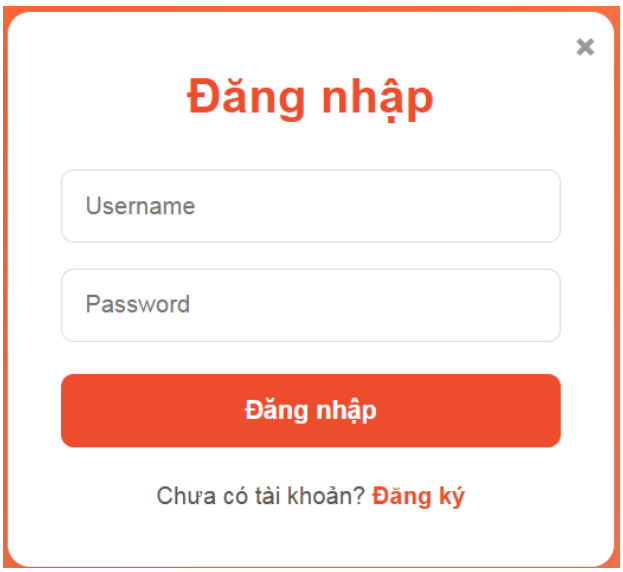
Đặc biệt, việc áp dụng các phương pháp Data Augmentation đã giúp cải thiện đáng kể hiệu suất mô hình. Điều này cho thấy vai trò quan trọng của việc tăng cường dữ liệu trong các bài toán gợi ý, đặc biệt khi dữ liệu có tính chất thưa và không đầy đủ.

Tuy nhiên, mô hình vẫn còn một số hạn chế và cần được cải thiện trong các nghiên cứu tiếp theo. Trong chương tiếp theo, mô hình sẽ được tích hợp vào hệ thống Web App nhằm minh họa khả năng gợi ý sản phẩm trong môi trường thực tế.

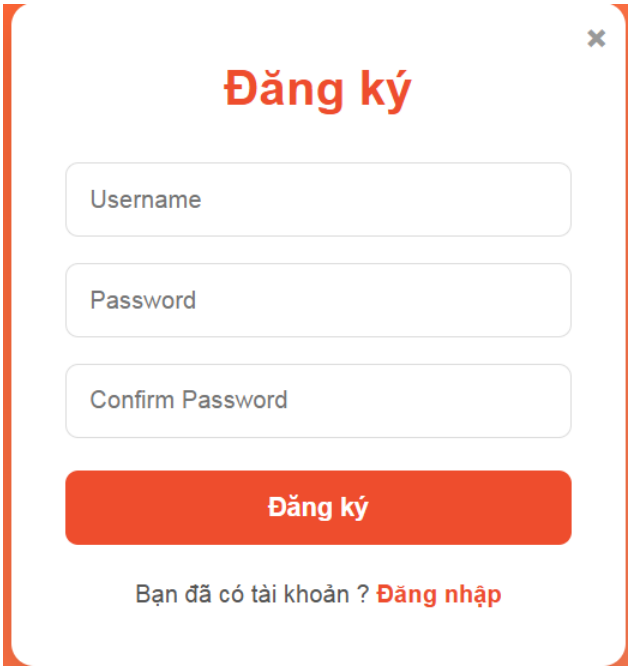
CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG KHAI PHÁ DỮ LIỆU

4.1. Triển khai các chức năng nghiệp vụ

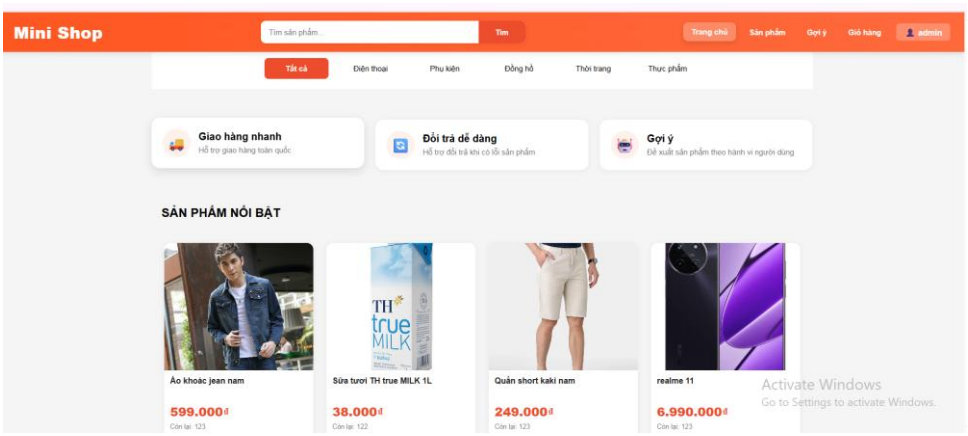
Sau khi hoàn thành quá trình phân tích và thiết kế hệ thống, nhóm tiến hành triển khai website bán hàng tích hợp hệ thống gợi ý sản phẩm dựa trên hành vi người dùng. Hệ thống được xây dựng theo mô hình web application sử dụng Flask cho phía backend, HTML/CSS/ JavaScript cho frontend và SQL Server để lưu trữ dữ liệu.



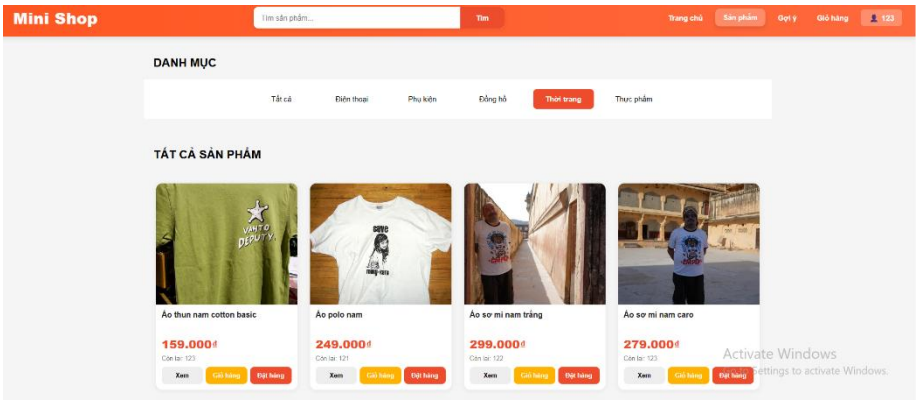
Hình 4. 1: Giao diện trang đăng nhập



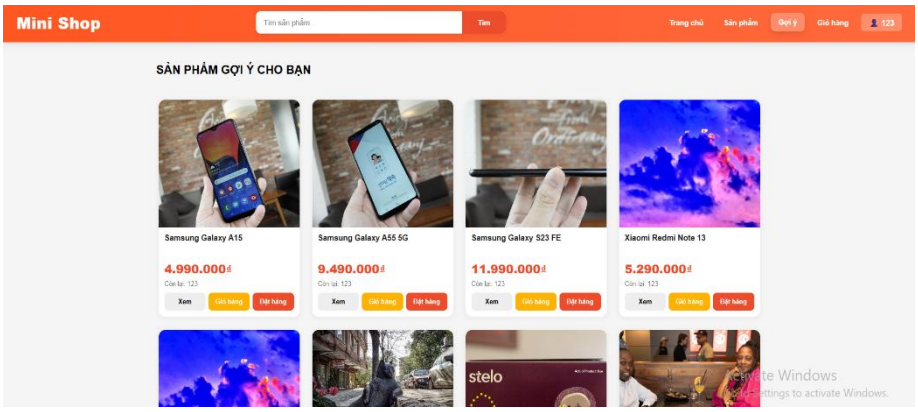
Hình 4. 2: Giao diện trang đăng ký



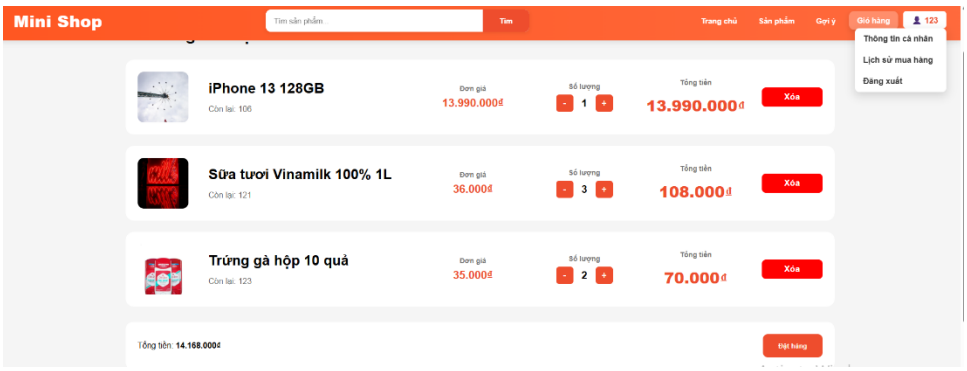
Hình 4. 3: Giao diện trang chủ



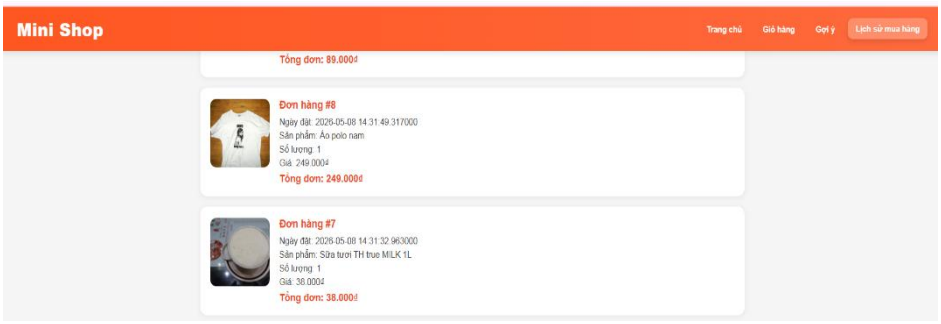
Hình 4. 4: Giao diện trang sản phẩm



Hình 4. 5: Giao diện trang gợi ý



Hình 4. 6: Giao diện trang giỏ hàng



Hình 4. 7: Giao diện trang lịch sử mua hàng

4.2. Kiểm thử đơn vị và triển khai hệ thống

Module API được kiểm thử nhằm đảm bảo frontend có thể giao tiếp ổn định với backend Flask. Các API được kiểm thử gồm:

- lấy danh sách sản phẩm,
- xem chi tiết sản phẩm,
- thêm vào giỏ hàng,
- đặt hàng,
- lấy lịch sử mua hàng,
- ghi nhận hành vi người dùng,
- lấy sản phẩm gợi ý.

Quá trình kiểm thử tập trung vào:

- kiểm tra request/response JSON,
- kiểm tra dữ liệu trả về đúng định dạng,

- kiểm tra xử lý lỗi và quyền truy cập khi chưa đăng nhập.

Việc kiểm thử giúp đảm bảo dữ liệu giữa frontend, backend, database và mô hình gợi ý được truyền chính xác và ổn định.

Quy trình xử lý của hệ thống được thực hiện theo các bước sau:

Người dùng truy cập website và thực hiện đăng nhập hoặc đăng ký tài khoản. Sau khi đăng nhập thành công, hệ thống lưu thông tin người dùng bằng session và hiển thị tên tài khoản trên thanh điều hướng.

Người dùng xem danh sách sản phẩm tại trang chủ hoặc trang sản phẩm. Khi người dùng bấm xem chi tiết, thêm vào giỏ hàng hoặc đặt hàng, frontend gửi request đến backend Flask để xử lý [32]. Đồng thời, hành vi của người dùng được lưu lại vào bảng events.

Khi người dùng thêm sản phẩm vào giỏ hàng, backend kiểm tra sản phẩm và số lượng tồn kho, sau đó lưu dữ liệu vào bảng giỏ hàng. Frontend cập nhật lại giao diện giỏ hàng, tổng tiền từng sản phẩm và tổng tiền toàn bộ giỏ hàng.

Khi người dùng đặt hàng, backend tạo đơn hàng mới, lưu thông tin chi tiết đơn hàng, cập nhật tồn kho và xóa sản phẩm tương ứng khỏi giỏ hàng nếu cần. Sau khi xử lý thành công, hệ thống hiển thị modal thông báo “Đặt hàng thành công”. Người dùng chỉ được chuyển sang trang lịch sử mua hàng sau khi bấm OK hoặc dấu X.

Đối với chức năng gợi ý sản phẩm, backend lấy lịch sử hành vi của người dùng từ bảng events, xây dựng chuỗi sản phẩm đã tương tác và đưa vào mô hình Sequential Recommendation. Mô hình trả về danh sách sản phẩm gợi ý, sau đó frontend render danh sách này tại trang “Gợi ý”.

Trong quá trình triển khai hệ thống, đề tài còn tồn tại một số thách thức và hạn chế nhất định.

Dữ liệu hành vi còn hạn chế

Dữ liệu hành vi trên website chủ yếu gồm các hành động như xem sản phẩm, thêm vào giỏ hàng và đặt hàng. Trong thực tế, hành vi người dùng còn có thể bao gồm thời gian xem, số lần quay lại sản phẩm, thao tác tìm kiếm hoặc lượt click vào danh mục. Việc chưa khai thác đầy đủ các tín hiệu này có thể ảnh hưởng đến chất lượng gợi ý.

Mô hình huấn luyện offline

Mô hình gợi ý hiện được huấn luyện offline. Khi có thêm dữ liệu hành vi mới, hệ thống chưa tự động cập nhật mô hình theo thời gian thực. Điều này khiến kết quả gợi ý có thể chưa phản ánh ngay sự thay đổi trong sở thích người dùng.

Độ trễ khi catalog sản phẩm lớn

Khi số lượng sản phẩm tăng lên, việc dự đoán và sắp xếp toàn bộ sản phẩm cho từng người dùng có thể làm tăng thời gian xử lý. Hệ thống cần tối ưu thêm bằng cách giới hạn tập ứng viên, cache kết quả phổ biến hoặc triển khai mô hình trên server mạnh hơn.

Chất lượng và dữ liệu sản phẩm

Một số dữ liệu sản phẩm được xây dựng phục vụ mục đích demo nên mô tả hoặc danh mục sản phẩm có thể chưa hoàn toàn đầy đủ như hệ thống thương mại điện tử thực tế. Đây là điểm cần được cải thiện nếu triển khai trong môi trường thật.

4.3. Kết chương

Đề tài được tiến hành triển khai và kiểm thử hệ thống website bán hàng tích hợp chức năng gợi ý sản phẩm dựa trên hành vi người dùng. Các chức năng chính như đăng ký, đăng nhập, quản lý thông tin cá nhân, giỏ hàng, đặt hàng, lịch sử mua hàng và gợi ý sản phẩm đã được xây dựng và hoạt động ổn định.

Bên cạnh đó, hệ thống cũng tích hợp mô hình Sequential Recommendation kết hợp Data Augmentation nhằm nâng cao chất lượng gợi ý sản phẩm. Kết quả kiểm thử cho thấy các thành phần frontend, backend, database và mô hình gợi ý có thể phối hợp và trao đổi dữ liệu chính xác, đáp ứng được các yêu cầu cơ bản của hệ thống.

TÀI LIỆU THAM KHẢO

[1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.

[2] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Burlington, MA, USA: Morgan Kaufmann, 2016.

[3] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[6] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds., *Recommender Systems Handbook*. New York, NY, USA: Springer, 2011.

[7] D. Jannach, M. Zanker, A. Felfernig, and G. Friedrich, *Recommender Systems: An Introduction*. Cambridge, U.K.: Cambridge University Press, 2010.

[8] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, “Recommender systems survey,” *Knowledge-Based Systems*, vol. 46, pp. 109–132, Jul. 2013, doi: 10.1016/j.knosys.2013.03.012.

[9] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, Jun. 2005, doi: 10.1109/TKDE.2005.99.

[10] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, “Evaluating collaborative filtering recommender systems,” *ACM Transactions on Information Systems*, vol. 22, no. 1, pp. 5–53, Jan. 2004, doi: 10.1145/963770.963772.

[11] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proceedings of the 10th International Conference on World Wide Web (WWW)*, Hong Kong, China, 2001, pp. 285–295, doi: 10.1145/371920.372071.

[12] G. Linden, B. Smith, and J. York, “Amazon.com recommendations: Item-to-item collaborative filtering,” *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, Jan.–Feb. 2003, doi: 10.1109/MIC.2003.1167344.

[13] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009, doi: 10.1109/MC.2009.263.

[14] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.

[15] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “BPR: Bayesian personalized ranking from implicit feedback,” in *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, Montreal, Canada, 2009, pp. 452–461.

[16] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.

[17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[18] F. A. Gers, J. Schmidhuber, and F. Cummins, “Learning to forget: Continual prediction with LSTM,” *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000, doi: 10.1162/089976600300015015.

[19] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proceedings of EMNLP*, Doha, Qatar, 2014, pp. 1724–1734.

[20] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014.

[21] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, “Session-based recommendations with recurrent neural networks,” in *Proceedings of ICLR*, San Juan, Puerto Rico, 2016.

[22] Y. K. Tan, X. Xu, and Y. Liu, “Improved recurrent neural networks for session-based recommendations,” in *Proceedings of DLRS@RecSys*, Boston, MA, USA, 2016, pp. 17–22, doi: 10.1145/2988450.2988452.

[23] W.-C. Kang and J. McAuley, “Self-attentive sequential recommendation,” in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, Singapore, 2018, pp. 197–206, doi: 10.1109/ICDM.2018.00035.

[24] F. Sun et al., “BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer,” in *Proceedings of CIKM*, Beijing, China, 2019, pp. 1441–1450, doi: 10.1145/3357384.3357895.

[25] X. Xie et al., “Contrastive learning for sequential recommendation,” *arXiv preprint arXiv:2010.14395*, 2020.

[26] P. Zhou et al., “Is contrastive learning necessary? A study of data augmentation vs. contrastive learning in sequential recommendation,” *arXiv preprint arXiv:2209.13902*, 2022.

[27] Y. Dang et al., “Uniform sequence better: Time interval aware data augmentation for sequential recommendation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[28] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[29] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[31] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[32] Pallets Projects, "Flask Documentation," 2026. [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Jun. 21, 2026].

[33] Retailrocket, "Retailrocket Recommender System Dataset," Kaggle, 2016. [Online]. Available: https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset. [Accessed: Jun. 21, 2026].